データベースを使用 Todo App

実装時間: 2.5 hours
Prisma ORM を使用したデータベース統合を学ぶ。SQLite を使用して、型安全なデータベースクエリを実装する。

実装されたもの

✅	Prisma ORM セットアップ	✅	データベースモデル
✅	SQLite データベース	✅	CRUD 操作
✅	マイグレーション	✅	Prisma Client 生成
✅	Express.js サーバー	✅	RESTful API
✅	型安全なクエリ	✅	リレーションシップ
✅	フィルタリング	✅	ソート機能

学んだこと

- Prisma ORM の基本
- データベースモデル定義
- マイグレーションの管理
- 型安全なデータベースクエリ
- リレーションシップの管理
- Express.js との統合
- RESTful API 設計

1. プロジェクト構造

実装内容: プロジェクトの初期化

プロジェクト構造

```
backend/
├── prisma/
│   ├── schema.prisma
│   ├── dev.db
│   └── migrations/
│       └── 20251101_init/
└── server.simple.mjs
```

パッケージ

パッケージ	バージョン	説明
prisma	5.22.0	Prisma CLI ツール
@prisma/client	5.22.0	Prisma Client ライブラリ
express	4.18.2	Web フレームワーク
cors	2.8.5	CORS ミドルウェア
nodemon	3.0.1	開発用ホットリロード

データベース

- SQLite: ファイルベースのデータベース
- 場所: prisma/dev.db
- 利点: セットアップ不要、ポータブル

2. データベースモデル

実装内容: Prisma スキーマでデータモデルを定義

prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = "file:./dev.db"
}

model User {
  id        Int      @id @default(autoinc)
  email     String   @unique
  name      String?
  createdAt DateTime @default(now)
  updated   DateTime @updated
  posts     Post[]
  profile   Profile?

  @table("users")
}

model Post {
  id        Int      @id @default(autoinc)
  title     String
  content   String?
  published Boolean  @default(false)
  authorId  Int
  author    User     @relation(fields: [authorId], references: [id])
  tags      Tag[]

  @table("posts")
}

model Profile {
  id        Int      @id @default(autoinc)
  bio       String?
  avatar    String?
  userId    Int      @unique
  user      User     @relation(fields: [userId], references: [id])

  @table("profiles")
}

model Tag {
  id        Int      @id @default(autoinc)
  name      String   @unique
  posts     Post[]

  @table("tags")
}
```

実装内容: User, Post, Profile, Tag モデル

リレーションシップ

モデル	リレーション	関連モデル	説明
User	posts	Post[]	1:N 投稿
User	profile	Profile?	1:1 プロフィール
Post	author	User	N:1 ユーザー
Post	tags	Tag[]	N:N タグ

データベース

```
$ npx prisma db push
Your SQLite database "dev.db" at "file:./dev.db"
is now created, schema is synced in 12ms

$ npx prisma generate
Generated, this will not install additional packages in the future.
✔ Generated Prisma Client (v5.22.0) to ./node_modules/@prisma/client in 1.2s
```

3. マイグレーション

実装内容: データベースマイグレーションの管理

マイグレーション

```
$ npx prisma migrate dev --name init
Applying migration `20251101_init`
The following migration(s) have been applied:
migrations/
  └─ 20251101_init/
    └─ migration.sql
Your database now in sync with your schema.
```

生成されたファイル

```
prisma/
├── dev.db
└── migrations/
    ├── 20251101_init/
    ├── migration_lock.toml
    └── 20251101_init/
        └── migration.sql
```

migration.sql

```
CREATE TABLE "users" (
"id" INTEGER NOT NULL PRIMARY KEY autoinc,
"email" TEXT NOT NULL,
"name" TEXT,
"createdAt" DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
"updated" DATETIME NOT NULL,
);
CREATE UNIQUE INDEX "users_email_unique" ON "users"("email");

CREATE TABLE "posts" (
"id" INTEGER NOT NULL PRIMARY KEY autoinc,
"title" TEXT NOT NULL,
"content" TEXT,
"published" BOOLEAN NOT NULL DEFAULT 0,
"authorId" INTEGER NOT NULL,
);

CREATE TABLE "profiles" (
"id" INTEGER NOT NULL PRIMARY KEY autoinc,
"bio" TEXT,
"avatar" TEXT,
"userId" INTEGER NOT NULL,
);
CREATE UNIQUE INDEX "profiles_userId_unique" ON "profiles"("userId");

CREATE TABLE "tags" (
"id" INTEGER NOT NULL PRIMARY KEY autoinc,
"name" TEXT NOT NULL,
);
CREATE UNIQUE INDEX "tags_name_unique" ON "tags"("name");
```

4. Prisma Client

実装内容: Prisma Client の生成

生成

```
$ npx prisma generate
Generated, this will not install additional packages in the future.
✔ Generated Prisma Client (v5.22.0) to ./node_modules/@prisma/client in 1.2s
```

使用方法

```
import { PrismaClient } from '@prisma/client'
const prisma = new PrismaClient()

// Create
const user = prisma.user.create({
  data: {
    email: 'alice@prisma.io',
    name: 'Alice',
    posts: {
      create: [
        { title: 'Hello World', published: true },
        { title: 'My first post', published: false }
      ],
    },
    profile: {
      create: { bio: 'I like turtles' },
    },
  }
})

// Read
const allUsers = prisma.user.findMany()
console.log(allUsers)

// Update
const updateUser = prisma.user.update({
  where: { email: 'alice@prisma.io' },
  data: { name: 'Alice Updated' },
})

// Delete
const deleteUser = prisma.user.delete({
  where: { email: 'bob@prisma.io' },
})

// Count
const count = prisma.user.count()

// Aggregate
const agg = prisma.post.aggregate({
  _avg: { authorId: true },
  _sum: { authorId: true },
  _count: { id: true },
  _max: { authorId: true },
  _min: { authorId: true },
})

// Group by
const groupPosts = prisma.post.groupBy({
  by: ['authorId'],
  _count: { id: true },
  having: { authorId: { gt: 10 } },
})

// Raw query
const result = prisma.$queryRaw`SELECT * FROM User WHERE id = ${1} OR email = ${'bob@prisma.io'};`

// Transaction
const [user1, user2] = prisma.$transaction([
  prisma.user.create({ data: { email: 'a@b.c' } }),
  prisma.user.create({ data: { email: 'b@c.d' } }),
])
```

5. サーバー統合

実装内容: Express.js サーバーに統合

server.simple.mjs

```
import express from 'express'
import cors from 'cors'
import { PrismaClient } from '@prisma/client'

const app = express()
const prisma = new PrismaClient()

app.use(cors())
app.use(express.json())

// GET /api/users
app.get('/api/users', async (req, res) => {
  const users = prisma.user.findMany({
    include: { posts: true, profile: true },
    orderBy: { createdAt: 'desc' },
  })
  res.json({ success: true, data: users, count: users.length })
})

// POST /api/users
app.post('/api/users', async (req, res) => {
  const { email, name, bio } = req.body
  const user = prisma.user.create({
    data: { email, name, profile: { create: { bio } } },
    include: { profile: true },
  })
  res.status(201).json({ success: true, data: user })
})

// GET /api/posts
app.get('/api/posts', async (req, res) => {
  const posts = prisma.post.findMany({
    include: { author: true, tags: true },
    orderBy: { createdAt: 'desc' },
    take: 10,
  })
  res.json({ success: true, data: posts, count: posts.length })
})

// POST /api/posts
app.post('/api/posts', async (req, res) => {
  const { title, content, authorId, tags, published } = req.body
  const post = prisma.post.create({
    data: { title, content, published, authorId, tags: { connect: tags.map(t => ({ name: t })) } },
    include: { author: true, tags: true },
  })
  res.status(201).json({ success: true, data: post })
})

// GET /api/stats
app.get('/api/stats', async (req, res) => {
  const [users, posts, tags, published] = prisma.$transaction([
    prisma.user.count(),
    prisma.post.count(),
    prisma.tag.count(),
    prisma.post.count({ where: { published: true } }),
  ])
  res.json({ success: true, data: { users, posts, tags, published } })
})

app.listen(3000, '0.0.0.0', () => {
  console.log('Server ready at http://0.0.0.0:3000')
})
```

エンドポイント

メソッド	パス	説明
GET	/api/users	ユーザー一覧
POST	/api/users	ユーザー作成
GET	/api/posts	投稿一覧
POST	/api/posts	投稿作成
GET	/api/stats	統計情報

6. CRUD 操作

実装内容: Create, Read, Update, Delete

Create

```
// Create user
POST /api/users
{
  "email": "alice@prisma.io",
  "name": "Alice",
  "bio": "I like turtles"
}

// Response
{
  "success": true,
  "data": {
    "id": 1,
    "email": "alice@prisma.io",
    "name": "Alice",
    "createdAt": "2025-11-01T...",
    "updated": "2025-11-01T...",
    "profile": {
      "id": 1,
      "bio": "I like turtles",
      "avatar": null,
      "userId": 1
    }
  }
}

// Create post
POST /api/posts
{
  "title": "Hello World",
  "content": "This is my first post",
  "authorId": 1,
  "tags": ["tech", "prisma"],
  "published": true
}

// Response
{
  "success": true,
  "data": {
    "id": 1,
    "title": "Hello World",
    "content": "This is my first post",
    "published": true,
    "authorId": 1,
    "author": { "id": 1, "email": "alice@prisma.io", "name": "Alice" },
    "tags": [ { "id": 1, "name": "tech" }, { "id": 2, "name": "prisma" } ]
  }
}
```

Read

```
// Get all users
GET /api/users

// Response
{
  "success": true,
  "data": [
    { "id": 1, "email": "alice@prisma.io", "name": "Alice", "posts": [], "profile": null },
    { "id": 2, "email": "bob@prisma.io", "name": "Bob", "posts": [], "profile": null }
  ],
  "count": 2
}

// Get all posts
GET /api/posts

// Response
{
  "success": true,
  "data": [
    { "id": 1, "title": "Hello World", "content": null, "published": false, "author": { "id": 1, "email": "alice@prisma.io", "name": "Alice" }, "tags": [] }
  ],
  "count": 1
}
```

Update

```
// Update user
PUT /api/users/1
{
  "name": "Alice Updated",
  "bio": "Updated bio"
}

// Response
{
  "success": true,
  "data": {
    "id": 1,
    "email": "alice@prisma.io",
    "name": "Alice Updated",
    "updated": "2025-11-01T...",
    "profile": { "id": 1, "bio": "Updated bio", "avatar": null, "userId": 1 }
  }
}

// Update post
PUT /api/posts/1
{
  "title": "Updated Title",
  "content": "Updated content",
  "published": true
}

// Response
{
  "success": true,
  "data": {
    "id": 1,
    "title": "Updated Title",
    "content": "Updated content",
    "published": true,
    "author": { "id": 1, "email": "alice@prisma.io", "name": "Alice Updated" },
    "tags": [ { "id": 1, "name": "tech" }, { "id": 2, "name": "prisma" } ]
  }
}
```

Delete

```
// Delete user
DELETE /api/users/2

// Response
{
  "success": true,
  "message": "User deleted"
}

// Delete post
DELETE /api/posts/1

// Response
{
  "success": true,
  "message": "Post deleted"
}
```

7. リレーションシップ

実装内容: 1:1, 1:N, N:N リレーションシップ

1:1 (User → Profile)

```
// Create user with profile
POST /api/users
{
  "email": "alice@prisma.io",
  "name": "Alice",
  "bio": "I like turtles"
}

// Response
{
  "success": true,
  "data": {
    "id": 1,
    "email": "alice@prisma.io",
    "name": "Alice",
    "profile": { "id": 1, "bio": "I like turtles", "avatar": null, "userId": 1 }
  }
}
```

1:N (User → Post)

```
// Create post
POST /api/posts
{
  "title": "Hello World",
  "content": "This is my first post",
  "authorId": 1,
  "tags": ["tech", "prisma"],
  "published": true
}

// Response
{
  "success": true,
  "data": {
    "id": 1,
    "title": "Hello World",
    "content": "This is my first post",
    "published": true,
    "author": { "id": 1, "email": "alice@prisma.io", "name": "Alice" },
    "tags": [ { "id": 1, "name": "tech" }, { "id": 2, "name": "prisma" } ]
  }
}
```

N:N (Post → Tag)

```
// Create post with tags
POST /api/posts
{
  "title": "Hello World",
  "content": "This is my first post",
  "authorId": 1,
  "tags": ["tech", "prisma"],
  "published": true
}

// Response
{
  "success": true,
  "data": {
    "id": 1,
    "title": "Hello World",
    "content": "This is my first post",
    "published": true,
    "author": { "id": 1, "email": "alice@prisma.io", "name": "Alice" },
    "tags": [ { "id": 1, "name": "tech" }, { "id": 2, "name": "prisma" } ]
  }
}
```

8. クエリ最適化

実装内容: 効率的なクエリ

include vs select

```
// include: 関連データを含める
const posts = prisma.post.findMany({
  include: { author: true, tags: true },
})

// select: 必要なフィールドのみ
const posts = prisma.post.findMany({
  select: { id: true, title: true, author: { select: { name: true } } },
})
```

N+1 問題を回避

```
// ❌ N+1 問題
const posts = prisma.post.findMany()
for (const post of posts) {
  const author = prisma.user.findUnique({ where: { id: post.authorId } })
}

// ✅ include で解決
const posts = prisma.post.findMany({
  include: { author: true },
})
```

インデックス

```
model User {
  id        Int      @id @default(autoinc)
  email     String   @unique
  name      String?
  createdAt DateTime @default(now)
  updated   DateTime @updated
  posts     Post[]
  profile   Profile?

  @table("users")
  @index([email])
  @index([name])
}

model Post {
  id        Int      @id @default(autoinc)
  title     String
  content   String?
  published Boolean  @default(false)
  authorId  Int
  author    User     @relation(fields: [authorId], references: [id])
  tags      Tag[]

  @table("posts")
  @index([authorId])
  @index([published])
  @index([createdAt])
}
```

ページネーション

```
// Page 1
const page1 = prisma.post.findMany({
  take: 10,
  skip: 0,
  orderBy: { createdAt: 'desc' },
})

// Page 2
const page2 = prisma.post.findMany({
  take: 10,
  skip: 10,
  orderBy: { createdAt: 'desc' },
})

// Cursor-based pagination
const cursorPosts = prisma.post.findMany({
  take: 10,
  cursor: { id: 100 },
  orderBy: { id: 'asc' },
})
```

9. トランザクション

実装内容: ACID トランザクション

$transaction

```
// Array transaction
const [user1, user2] = prisma.$transaction([
  prisma.user.create({ data: { email: 'a@b.c' } }),
  prisma.user.create({ data: { email: 'b@c.d' } }),
])

// Interactive transaction
const result = prisma.$transaction(tx => {
  const user = tx.user.create({ data: { email: 'x@y.z' } })
  const post = tx.post.create({ data: { title: 'Test', authorId: user.id } })
  return { user, post }
})
```

ロールバック

```
// エラーが発生すると自動ロールバック
const result = prisma.$transaction(tx => {
  const user = tx.user.create({ data: { email: 'x@y.z' } })
  const post = tx.post.create({ data: { title: 'Test', authorId: 999 } // 存在しない
  return { user, post }
})
// → user も post も作成されない
```

10. 集計クエリ

実装内容: count, aggregate, groupBy

count

```
// 全ユーザー数
const count = prisma.user.count()

// 条件付き
const publishedCount = prisma.post.count({ where: { published: true } })
```

aggregate

```
const agg = prisma.post.aggregate({
  _avg: { authorId: true },
  _sum: { authorId: true },
  _count: { id: true },
  _max: { authorId: true },
  _min: { authorId: true },
})
```

groupBy

```
const groupPosts = prisma.post.groupBy({
  by: ['authorId'],
  _count: { id: true },
  having: { authorId: { gt: 10 } },
})
```

11. Raw Queries

実装内容: 生の SQL クエリ

$queryRaw

```
const result = prisma.$queryRaw`SELECT * FROM User WHERE id = ${1} OR email = ${'bob@prisma.io'};`
```

$executeRaw

```
const result = prisma.$executeRaw`UPDATE User SET name = ${'Updated'} WHERE id = ${1};`
```

12. データベース管理

実装内容: Prisma Studio, db push, db pull

Prisma Studio

```
$ npx prisma studio
Prisma Studio is up on http://localhost:5555
```

db push

```
$ npx prisma db push
Your SQLite database "dev.db" at "file:./dev.db"
is now created, schema is synced in 12ms
```

db pull

```
$ npx prisma db pull
Introspecting based on datasource defined in prisma/schema.prisma
Your schema has been updated.
```

migrate reset

```
$ npx prisma migrate reset
Reset the database? (y/N) y
Applying migration `20251101_init`
Your database now in sync with your schema.
```

13. Seeding

実装内容: データベースシード

prisma/seed.mjs

```
import { PrismaClient } from '@prisma/client'
const prisma = new PrismaClient()

const users = [
  { email: 'alice@prisma.io', name: 'Alice', bio: 'I like turtles' },
  { email: 'bob@prisma.io', name: 'Bob', bio: 'I like cats' },
]

for (const u of users) {
  const user = prisma.user.create({
    data: { email: u.email, name: u.name, profile: { create: { bio: u.bio } } },
  })
  console.log(`Created user: ${user.email}`)
}

const post = prisma.post.create({
  data: { title: 'Hello World', content: 'This is my first post', published: true, author: { connect: { email: 'alice@prisma.io' } } },
}
console.log(`Created post: ${post.title}`)

const tag = prisma.tag.create({
  data: { name: 'tech' }
}
console.log(`Created tag: ${tag.name}`)

prisma.$disconnect()
```

package.json

```
"prisma": {
  "seed": "node prisma/seed.mjs"
}
```

実行

```
$ npx prisma db seed
Seed command has been configured in package.json
Running seed command `node prisma/seed.mjs` ...
Created user: alice@prisma.io
Created user: bob@prisma.io
Created post: Hello World
Created tag: tech
The seed command has been executed.
```

14. エラーハンドリング

実装内容: Prisma エラーの処理

エラーコード

コード	説明
P2002	Unique constraint violation
P2003	Foreign key constraint violation
P2004	Constraint violation
P2005	Invalid value for field
P2006	Invalid value for field
P2007	Data validation error
P2008	Query parsing error
P2009	Query validation error
P2010	Raw query failed
P2011	Null constraint violation
P2012	Missing required value
P2013	Missing required argument
P2014	Relation violation
P2015	Related record not found
P2016	Query interpretation error
P2017	Records for relation not connected
P2018	Connected records not found
P2019	Input error
P2020	Value out of range
P2021	Table does not exist
P2022	Column does not exist
P2023	Inconsistent column data
P2024	Timed out fetching a new connection from the connection pool
P2025	Operation failed because it depends on one or more records that were required but not found.
P2026	The current database provider doesn't support a feature that the schema uses.
P2027	Multiple errors occurred on the database during migration
P2028	Transaction API error
P2029	Query parameter limit exceeded in a prepared statement
P2030	Cannot find a fulltext index to use for the search
P2031	Prisma Client could not connect to the MongoDB Replica Set. Please check the connection string.
P2032	Error in MongoDB connection string
P2033	Error in MongoDB connection string
P2034	Transaction failed due to a write conflict or a deadlock. Please retry your transaction
P2035	Assertion violation on the database
P2036	Error in MongoDB connection string
P2037	Too many connections to the database

実装

```
// Error handling
try {
  const user = prisma.user.create({ data: { email: 'alice@prisma.io' } })
} catch (e) {
  if (e instanceof Prisma.PrismaClientKnownRequestError) {
    console.log(e.code)
    console.log(e.meta)
    console.log(e.message)
  }
}

// Specific error codes
try {
  const user = prisma.user.create({ data: { email: 'alice@prisma.io' } })
} catch (e) {
  if (e.code === 'P2002') {
    console.log('Unique constraint violation')
    console.log('Field:', e.meta.target)
  }
  if (e.code === 'P2003') {
    console.log('Foreign key constraint violation')
    console.log('Field:', e.meta.field_name)
  }
  if (e.code === 'P2025') {
    console.log('Record not found')
    console.log('Cause:', e.meta.cause)
  }
}

// Validation errors
try {
  const user = prisma.user.create({ data: { email: 'invalid' } })
} catch (e) {
  if (e.code === 'P2009') {
    console.log('Validation error')
    console.log('Message:', e.message)
  }
}
```

15. パフォーマンス

実装内容: 最適化テクニック

N+1 問題

```
// ❌ N+1 問題
const posts = prisma.post.findMany()
for (const post of posts) {
  const author = prisma.user.findUnique({ where: { id: post.authorId } })
}

// ✅ include で解決
const posts = prisma.post.findMany({
  include: { author: true },
}
```

インデックス

```
model User {
  id        Int      @id @default(autoinc)
  email     String   @unique
  name      String?
  createdAt DateTime @default(now)
  updated   DateTime @updated
  posts     Post[]
  profile   Profile?

  @table("users")
  @index([email])
  @index([name])
}

model Post {
  id        Int      @id @default(autoinc)
  title     String
  content   String?
  published Boolean  @default(false)
  authorId  Int
  author    User     @relation(fields: [authorId], references: [id])
  tags      Tag[]

  @table("posts")
  @index([authorId])
  @index([published])
  @index([createdAt])
}
```

ページネーション

```
// Page 1
const page1 = prisma.post.findMany({
  take: 10,
  skip: 0,
  orderBy: { createdAt: 'desc' },
}

// Page 2
const page2 = prisma.post.findMany({
  take: 10,
  skip: 10,
  orderBy: { createdAt: 'desc' },
}

// Cursor-based pagination
const cursorPosts = prisma.post.findMany({
  take: 10,
  cursor: { id: 100 },
  orderBy: { id: 'asc' },
}
```

select vs include

```
// include: 関連データを含める
const posts = prisma.post.findMany({
  include: { author: true, tags: true },
}

// select: 必要なフィールドのみ
const posts = prisma.post.findMany({
  select: { id: true, title: true, author: { select: { name: true } } },
}
```

16. 実装サマリー

実装内容: 完了した機能

✅ 完了した機能

- Prisma ORM セットアップ
- SQLite データベース
- マイグレーション
- Prisma Client 生成
- Express.js サーバー
- RESTful API
- 型安全なクエリ
- リレーションシップ
- フィルタリング
- ソート機能
- CRUD 操作
- 1:1 リレーション
- 1:N リレーション
- N:N リレーション
- 集計クエリ
- トランザクション
- Raw Queries
- Seeding
- エラーハンドリング
- パフォーマンス最適化

📊 統計

モデル	4
エンドポイント	5
リレーション	3
実装時間	2.5 hours

🎯 学んだこと

- Prisma ORM の基本
- データベースモデル定義
- マイグレーションの管理
- 型安全なデータベースクエリ
- リレーションシップの管理
- Express.js との統合
- RESTful API 設計